

HelixTrack

HelixTrack

JIRA 自由世界的替代方案

概要

HelixTrack 是一款全面、现代的开源解决方案，可替代 JIRA（并通过其 Documents 扩展替代 Confluence）——一套基于 Go 微服务后端构建的多平台项目管理与问题跟踪系统，拥有原生 Web、桌面及移动端客户端。

简介

开源的 JIRA/Confluence 替代方案。基于 Go 微服务后端（“HelixTrack Core”）提供统一的 REST API 接口，支持项目与问题跟踪，并提供类似 Confluence 的文档工作区，通过 HTTP/3 QUIC 协议为原生 Web、桌面、Android 及 iOS 客户端提供服务。

详细介绍

HelixTrack 是一款开源的项目管理与问题跟踪平台，定位为 JIRA 和 Confluence 的自由世界替代方案——重新构建了大多数工程团队依赖的两款产品，成为一套完全自主可控、可部署于任何环境的软件。其核心是 **HelixTrack Core**，一套采用 Go 语言和 Gin 框架开发的 REST API 微服务，提供完整的问题跟踪、敏捷/Scrum 看板、团队管理及分层权限引擎。该权限引擎的实现可在本地进程内引擎与 HTTP 后端服务之间灵活切换，因此同一授权模型能够从单台笔记本电脑扩展至分布式集群，无需修改应用代码。Core 并未将 REST 接口分散至数十个路由，而是通过单一的动作路由 /do 端点统一处理所有请求，采用一致的请求/响应封装（action/jwt/object/data 输入，errorCode/errorMessage/data 输出）。所有客户端遵循同一简洁契约，新增功能仅需添加动作，而非新增需要文档化、安全管理及版本控制的 URL。Core 与解耦的认证、权限及本地化服务集成，这些服务通过 HTTP/3 QUIC 协议通信，可部署于独立机器或集群，亦可在测试环境中完全关闭。数据存储于 SQLite（零配置开发环境）及 PostgreSQL（生产环境），并默认采用 SQLCipher（AES-256）加密静态数据，确保敏感项目数据在磁盘上得到保护，而非事后补救。**Documents V2** 扩展将问题跟踪

器升级为完整的知识平台：提供类似 Confluence 的工作区，包含空间、页面、版本控制、模板、实时 WebSocket 协作及分析功能——终于将 Wiki 与问题跟踪器整合至同一后端，而非两款拼凑的产品。围绕 Core 构建的多个客户端应用包括：Angular Web 客户端、Tauri + Angular 桌面客户端、原生 Android (Kotlin) 及 iOS (Swift) 应用，以及鸿蒙系统和 Aurora OS 客户端，甚至一款屏保程序——所有客户端均与同一后端通信，并通过 UDP 广播在局域网内自动发现服务器，无需手动配置。客户端应用以独立私有仓库维护，此处仅从产品层面呈现。

我们为何构建它

为了给团队提供一个真正开放、可自托管的替代方案，取代 JIRA + Confluence 这一套工具组合——「为了自由的世界」——摆脱供应商锁定，将企业级任务跟踪、文档管理与协作功能整合于同一个开源许可之下。

为何它是颠覆性的

它将两大重量级商业产品——问题跟踪系统与 Wiki/文档管理套件——整合为一个开放、高性能且可自托管的平台，并提供了现有厂商从未提供的功能：真正跨平台的原生客户端（Web、桌面、Android、iOS，以及 HarmonyOS 和 Aurora OS），且全部由同一后端契约驱动。其核心优势在于「无妥协的所有权」。HTTP/3 无处不在的全解耦微服务架构，加上静态数据的 SQLCipher AES-256 加密，将原本仅属于专有 SaaS 的性能与安全标准带入自托管系统——无需席位许可、无供应商锁定、数据永不离开你的基础设施。团队能在自有硬件上，以单一开源许可获得他们熟悉的 JIRA + Confluence 体验。

创新之处

- 统一的基于动作的 /do API 接口——一个端点、一个封装，按动作路由。新功能以新动作形式呈现，而非新 URL，将攻击面、客户端代码及文档负担压缩至单一共享契约。
- HTTP/3 QUIC 作为默认服务间通信协议——从第一天起即采用现代低延迟、连接弹性的网络架构，而非后期补丁。
- 可在本地进程内实现与 HTTP 后端服务间自由切换的权限引擎，同时支持可选的独立部署 Auth、Permissions 及 Localization 服务——无论单进程还是集群，均采用相同的授权模型。
- 通过 --space-root 标志实现多空间数据隔离：每个项目拥有独立的数据库与资产存储，租户与项目在存储层面而非查询过滤器层面实现隔离。
- 静态数据的 SQLCipher AES-256 加密——敏感项目数据在磁盘上默认透明加密。
- 通过 UDP 广播实现客户端与服务器的自动发现——客户端无需手动配置即可在局域网内找到 Core。
- 文档 V2，真正的「Confluence 替代方案」，支持乐观锁并行编辑、冲突检测及完整变更历史——真正的协同文档，与任务跟踪器共享同一后端。

最大的技术挑战及解决方案

- 六大客户端平台，一个后端，零契约偏移。维护 Web/Angular、桌面/Tauri、Android/Kotlin、iOS/Swift、HarmonyOS 及 Aurora 客户端通常意味着六套发散的 API 集成，且极易失去同步。我们通过将单一动作路由的 /do API 及其固定封装作为唯一契约，消除了这一风险——所有客户端均以相同方式对接，并在此基础上叠加 UDP 广播服务发现，使客户端无需手动配置端点即可定位 Core。
- 解耦服务而不牺牲延迟。将 Auth、Permissions 及 Localization 拆分为独立部署的服务通常会增加每次调用的网络跳数。我们在所有服务间调用中采用 HTTP/3 QUIC，确保跳数快速且连接弹性，并使每个服务均可独立运行——甚至在测试配置中完全禁用——因此解耦是一种部署选择，而非固定成本。
- 实现 **Confluence** 级别的协作，避免写入冲突。实时多人编辑极易导致写入冲突。文档 V2 通过乐观锁机制下的空间/页面/版本控制，结合显式冲突检测、完整变更历史及实时 WebSocket 同步，解决了这一问题——协作保持一致，而非悄然覆盖编辑内容。
- 静态数据加密而不牺牲吞吐量。SQLCipher AES-256 保护磁盘数据，但会增加每次查询的开销；我们通过多层缓存（Localization 服务中的内存 LRU 缓存 + Redis）进行优化，使多语言查找等热点路径保持高速，同时确保数据始终加密。

内容

技术栈

- **Go + Gin** —— 选用此组合构建高吞吐、低延迟的 HTTP 服务，支持单二进制部署；内置 Core 的 REST API 框架、JWT/CORS 中间件，以及作为整个系统前端的基于动作路由的 /do 路由器。
- **HTTP/3 QUIC** —— 选用此协议作为 Core 与认证/权限/本地化服务之间的传输层，因其多路复用及连接迁移设计能有效降低尾延迟，并在网络不稳时保持稳定，而 TCP 则可能出现卡顿。
- **PostgreSQL**（生产环境）/ **SQLite**（开发环境） —— 单一关系模型支撑跨两种引擎的大规模追踪与文档架构：SQLite 确保本地开发零配置且基于文件存储，而 Postgres 则通过专用的 production Compose 配置文件接管生产环境的扩展需求。
- **SQLCipher (AES-256)** —— 选用此方案实现透明的数据库级静态加密，保护敏感项目数据无需应用层加密，且无需修改查询写法。
- **Redis** —— 选用其作为本地化服务的共享缓存层，位于内存 LRU 缓存之后，构成双层缓存架构，确保即使底层存在加密开销，热门多语言查询依然保持高速响应。
- **Uber Zap + Lumberjack** —— 选用此组合实现结构化、低内存分配的日志记录，内置日志轮转功能，确保 Core 在生产环境中可观测性不受无限增长的日志影响。
- **golang-jwt / JWT** —— 选用无状态认证机制，签名令牌嵌入每个 /do 请求的 jwt 字段，确保所有客户端的认证方式统一。
- **Angular 19**（搭配 **Material**、**RxJS**） —— 选用此框架构建响应式、组件驱动的浏览器客户端，内置成熟的 Material 设计系统。

- **Tauri 2.0 + Rust + Angular** —— 选用此组合打造原生桌面应用外壳，通过复用 Angular UI 并嵌入 Rust 支持的 WebView，避免捆绑完整浏览器运行时，从而实现极小的安装包体积。
- **Kotlin (Android) / Swift + SwiftUI (iOS)** —— 选用此方案为移动用户提供真正原生、符合平台设计规范的客户端，而非封装的 WebView。
- **Docker / Docker Compose** (兼容 **Podman**) —— 选用此方案实现可复现的容器化部署，内置 /health 健康检查，且兼容 Podman，无需依赖特定守护进程或供应商。
- **Testify (Go) ; Cypress/Playwright/Karma+Jasmine** (客户端) —— 选用分层自动化测试方案，独立覆盖后端接口契约与客户端 UI，与「单一后端/多客户端」架构相匹配。

状态与诚信说明

- 状态: **Beta** 版。HelixTrack Core 是一个可运行的 REST API 微服务；文档 **V2** 扩展功能已完成约 95%，但存在已知的数据库字段映射问题，因此尚未作为正式发布版本提供。
- 许可证: 待定。CLAUDE.md 文件声明为 MIT 许可证，但 core/LICENSE 记录文件则为 Apache 2.0 —— 这一矛盾需在最终确定许可证前解决。
- 项目 README 中提及的性能数据（如每秒 50,000+ 请求、亚毫秒级查询时间）均为设计/营销目标，而非独立发布的基准测试结果，因此上述说明中未包含此类声明。
- 客户端应用（Web、桌面、Android、iOS、Aurora、HarmonyOS）均存放于私有代码库中，此处仅从产品层面进行描述。

优先级别: Helix-主线项目暨Helix-Track产品线的旗舰项目，优先级高于所有Server Factory项目。